Universität Rostock

OBJEKTE — IMPLEMENTIERUNG

Joern Ploennigs · AI4SC

Midjourney: Bauhaus line drawing

Zielsetzung

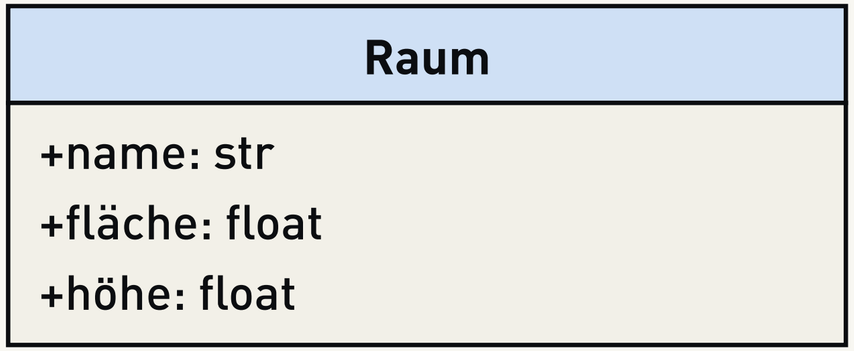
- UML-Klassendiagramm aus S04 in Python-Syntax übersetzen
- `class`, `__init__`, `self`, Attribute und Methoden aus dem Entwurf lesen
- KI-generierten Klassencode gegen das UML-Diagramm verifizieren
- Typische KI-Fehler bei Klassen erkennen und korrigieren

VON UML ZU PYTHON

Das Mapping

UML-Element	Python-Syntax
Klassenname `Raum`	`class Raum:`
Attribut `name: str`	`self.name = name` in `__init__`
Methode `f() → float`	`def f(self) -> float:`
Assoziation `→ Fenster (0..*)`	`self.fenster = []` als Instanzattribut

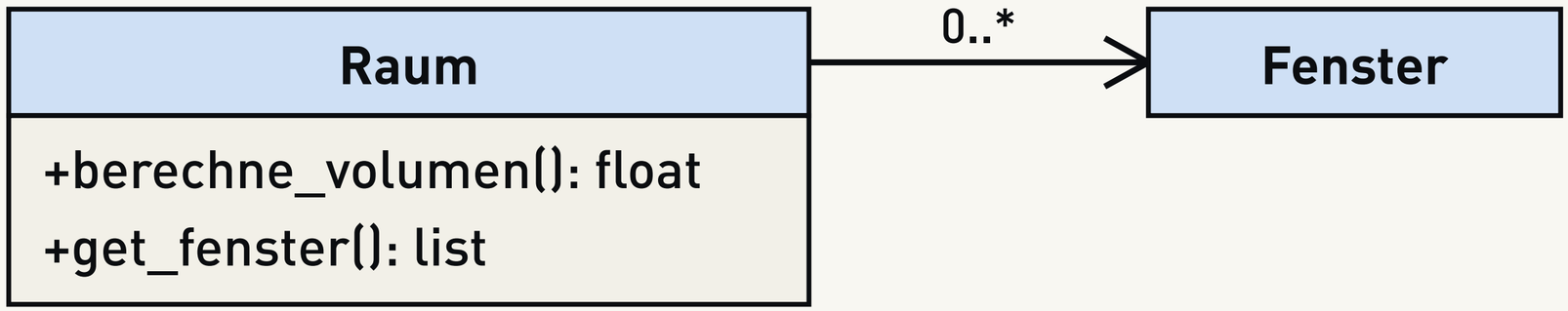
Klassenname und Attribute




```python
class Raum:
    def __init__(self,
                 name: str,
                 fläche: float,
                 höhe: float):
        self.name   = name
        self.fläche = fläche
        self.höhe   = höhe
```

Eine Klasse wird zu `class Raum:`, jedes Attribut zu `self.attribut` in `__init__`.

Methode und Assoziation




```python
class Raum:
    def __init__(self, ...):
        ...
        self.fenster = []

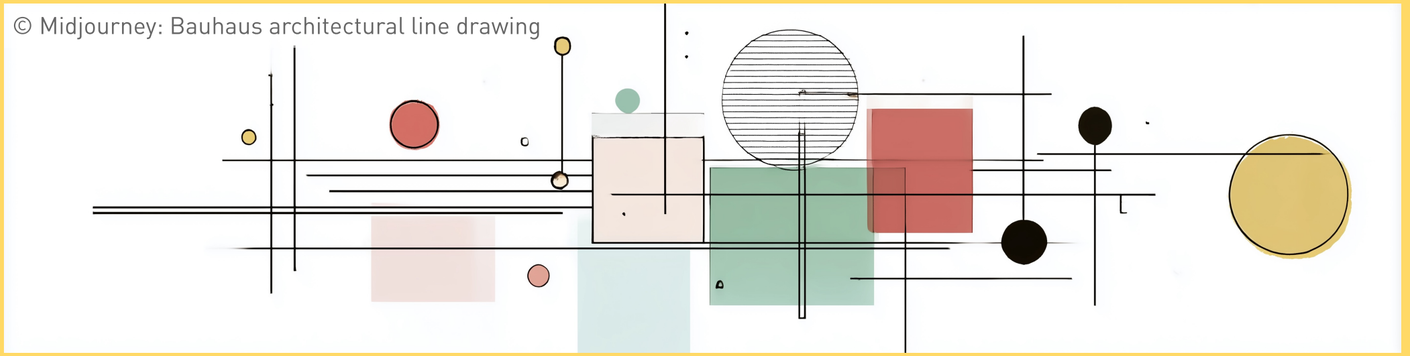
    def berechne_volumen(self) -> float:
        return self.fläche * self.höhe

    def get_fenster(self):
        return self.fenster
```

- Methode → `def … (self)`
- Assoziation → Instanzattribut
- `0..*` → leere Liste in `__init__`

Hörsaalfrage

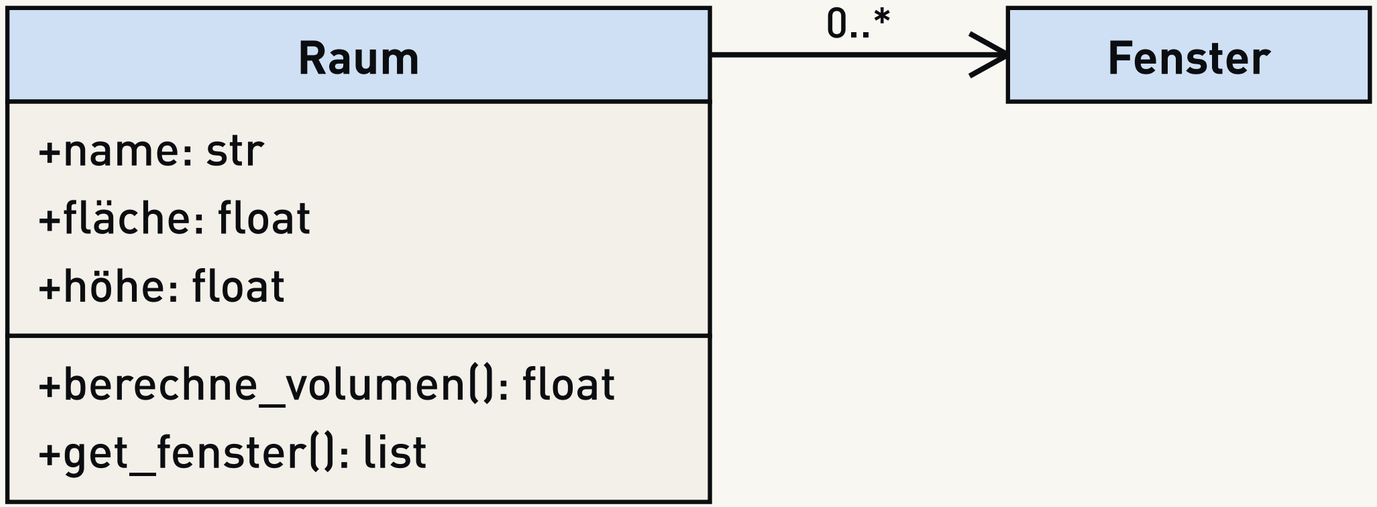
Welche Python-Klasse entspricht dem UML-Diagramm der Raum-Klasse aus S04?



BIM-BEISPIEL: RAUM-KLASSE

KI generiert die Raum-Klasse

Prompt: *„Implementiere die Klasse `Raum` aus diesem UML-Diagramm."*




```python
# KI-generiert
class Raum:
    def __init__(self, name: str, fläche: float, höhe: float):
        self.name   = name
        self.fläche = fläche
        self.höhe   = höhe
        self.fenster = []

    def berechne_volumen(self) -> float:
        return self.fläche * self.höhe

    def get_fenster(self):
        return self.fenster
```

Vor dem Ausführen: Was gibt `berechne_volumen()` zurück? Was gibt `get_fenster()` zurück? Welche Anforderung ist noch nicht erfüllt?

AI4 SMART CONSTRUCTION

Verifizieren gegen das Diagramm

Agentic Workflow — Klasse verifizieren:

01 Ist jedes Attribut aus dem UML-Diagramm in `__init__` vorhanden?

02 Hat jede Methode die richtige Signatur (Parameter, Rückgabetyp)?

03 Ist jede Assoziation als Instanzattribut implementiert?

04 Ist Kapselung dort umgesetzt, wo das Diagramm sie fordert?

PYTHON-KLASSEN — SYNTAX IM DETAIL

Konstruktor `__init__`

- `__init__()` ist die spezielle Methode zum Erzeugen einer Instanz
- Wird implizit aufgerufen wenn der Klassenname wie eine Funktion genutzt wird
- Weist Instanzattribute zu und führt Initialisierungsschritte aus
- `self` = Selbstreferenz auf die neue Instanz (immer erster Parameter)

```python
class Punkt:
    def __init__(self, x: float, y: float):
        self.x = x
        self.y = y
```

Instanz- vs. Klassenattribute

Instanzattribute (Wert je Instanz):

```python
class Raum:
    def __init__(self, fläche: float):
        self.fläche = fläche  # je Instanz
```

Klassenattribute (Wert für alle):

```python
class Raum:
    einheit = "m²"  # alle Instanzen teilen diesen Wert

    def __init__(self, fläche: float):
        self.fläche = fläche
```

> ⚠ **Achtung**
>
> Veränderliche Klassenattribute (Listen, Dicts) gelten für **alle** Instanzen — eine Änderung durch eine Instanz wirkt sich auf alle aus.

Methoden

- Methoden werden wie Funktionen mit `def` definiert — auf Klassenebene eingerückt
- Erster Parameter ist immer `self` (Referenz auf die aktuelle Instanz)
- Zugriff auf Attribute der Instanz über `self.attributname`
- Man kann Parametern auch Defaults zuweisen

```python
class Raum:
    def __init__(self, fläche: float, höhe: float):
        self.fläche = fläche
        self.höhe   = höhe

    def berechne_volumen(self) -> float:
        return self.fläche * self.höhe
```

Instanzen erzeugen und nutzen

- Instanzen werden erzeugt indem der Klassenname als Funktion aufgerufen wird
- `self` wird dabei von Python automatisch übergeben
- Benannte Parameter empfohlen — vermeiden semantische Unklarheiten
- Attributzugriff und Methodenaufruf über den Punkt-Syntax

```
raum = Raum(name="Büro 1", fläche=30.0, höhe=2.8)
print(raum.fläche)              # 30.0
print(raum.berechne_volumen()) # 84.0
```

ASSOZIATIONEN IMPLEMENTIEREN

Assoziationen als Instanzattribute

Eine ⇆ Assoziation zwischen Klassen wird als Instanzattribut implementiert:

```python
class Raum:
    def __init__(self, name: str, fläche: float, höhe: float):
        self.name    = name
        self.fläche  = fläche
        self.höhe    = höhe
        self.fenster = []  # ⇄ Assoziation → Fenster (0..*)

    def add_fenster(self, fenster):
        self.fenster.append(fenster)
```

- `0..*` → leere Liste (`[]`), wächst mit `append`
- `1` → direktes Attribut, Pflichtparameter im Konstruktor
- `0..1` → optionales Attribut, Default `None`

KAPSELUNG

Kapselung: Attribute schützen

- Private Attribute: `__attributname` — nur innerhalb der Klasse sichtbar
- Protected Attribute: `_attributname` — für Klasse und Unterklassen
- Public Attribute: `attributname` — überall sichtbar (Standard)
- Getter-Methoden stellen kontrollierten Lesezugriff bereit
- Python nutzt bei `__name` Name Mangling: technisch bleibt Zugriff möglich, aber nicht unter dem ursprünglichen Namen

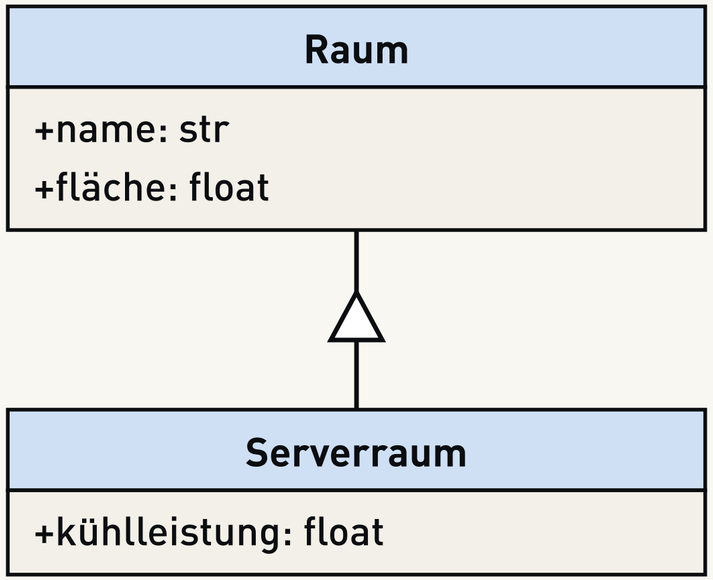
```python
class Raum:
    def __init__(self, fläche: float):
        self.__fläche = fläche  # privat – von außen nicht direkt änderbar

    def get_fläche(self) -> float:
        return self.__fläche
```

VERERBUNG

Vererbung

- Unterklassen erben alle Attribute und Methoden der Oberklasse
- In UML: Generalisierungspfeil (weißer Dreieckspfeil)
- In Python: Oberklasse in Klammern beim `class`-Schlüsselwort
- `super().__init__(...)` ruft den Konstruktor der Oberklasse auf




```python
class Raum:
    def __init__(self, name: str, fläche: float):
        self.name   = name
        self.fläche = fläche

class Serverraum(Raum):
    def __init__(self, name: str, fläche: float, kühlleistung: float):
        super().__init__(name, fläche)   # Elternkonstruktor
        self.kühlleistung = kühlleistung
```

TYPISCHE KI-FEHLER BEI KLASSEN

KI-Fehler 1 — Mutable Default in `__init__`

Fehlerhaft (KI macht das oft):

```
class Raum:
    def __init__(self,
                 fenster=[]):  # BUG
        self.fenster = fenster
```

Alle Instanzen **teilen dieselbe Liste** — eine Änderung wirkt sich auf alle aus.

Richtig:

```
class Raum:
    def __init__(self,
                 fenster=None):
        if fenster is None:
            fenster = []
        self.fenster = fenster
```

Jede Instanz bekommt ihre **eigene Liste**.

> **Merke**
>
> Das Problem gilt nicht nur in `__init__`, sondern bei **jeder** Python-Funktion mit veränderlichem Defaultwert.

KI-Fehler 2 — Fehlendes `self`

Fehlerhaft:

```
class Raum:
    def berechne_volumen():  # BUG
        return fläche * höhe
```

Kein `self` → kein Zugriff auf Instanzattribute, `TypeError` beim Aufruf.

Richtig:

```
class Raum:
    def berechne_volumen(self):
        return self.fläche * self.höhe
```

`self` macht den Instanzkontext explizit.

KI-Fehler 3 — `cls` statt `self`

Fehlerhaft:

```python
class Raum:
    @classmethod
    def berechne_volumen(cls):  # BUG
        return cls.fläche * cls.höhe
```

`cls` ist die **Klasse**, nicht die Instanz: `Raum().berechne_volumen()` löst einen `AttributeError` aus, denn `fläche` und `höhe` gehören zur Instanz.

Richtig:

```python
class Raum:
    def berechne_volumen(self):
        return self.fläche * self.höhe
```

Instanzmethode → immer `self`, ohne Dekorator.

Merksatz

`self` = Instanzmethode · `cls` = `@classmethod` · kein Parameter = `@staticmethod`

CODE-VERSTÄNDNIS-CHECKLISTE

Drei Fragen für KI-generierten Klassencode

1. Was modelliert diese Klasse?

- Welche Entität oder welches Konzept wird abgebildet?
- Passt der Klassenname zur Domäne?

2. Welche Anforderung implementiert sie?

- Gibt es ein UML-Diagramm oder eine Spezifikation?
- Sind alle Attribute und Methoden aus dem Entwurf vorhanden?

3. Was könnte schiefgehen?

- Gibt es mutable Defaults in `__init__`?
- Fehlt `self` bei einer Methode?
- Sind Assoziationen als Instanzattribute implementiert?

Querverbindungen

S04 — `04c_Objects`

UML als Entwurfsartefakt:
Klassendiagramm, Beziehungen, Multiplizität

S06 — `06c_Module` (folgt)

Klassen in Module organisieren:
`import`, Paketstruktur, Namespaces

S08 — `08b_CodeReview`

OOP Best Practices in der Review-Checkliste:
Kapselung, Vererbung, Methodensignaturen

QUESTIONS